Ders 5 Genişletilmiş Notu: Paketler, Sarmalayıcı Sınıflar, String, Math ve Random, Scanner, Parametre Aktarımı ve toString

Ders 5 Genişletilmiş Notu: Paketler, Sarmalayıcı Sınıflar, String, Math ve Random, Scanner, Parametre Aktarımı ve toString

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bir Düzeltme ve Önceki Derslerin Özeti
- 2. Paketler ve import
- 3. Bilgi Gizleme: Üyeleri Neden Gizliyoruz?
- 4. UML'de Erişim Sembolleri ve Yazım Düzeni
- 5. final ve static Tekrarı
- 6. Tam Sayı Tipleri ve Tür Dönüşümü
- 7. İlk Swap Denemesi: Integer ile Çalışmıyor
- 8. Diğer İlkel Tipler ve İlkel Veri Üzerinde İşlemler
- 9. Sarmalayıcı Sınıflar
- 10. String Sınıfı
- 11. Kendi Sarmalayıcı Sınıfımız: Swap'i Çalıştırmak
- 12. Math Sınıfı
- 13. Sözde Rastgele Sayılar ve Random Sınıfı
- 14. Çıktı: System.out
- 15. Girdi: Scanner ve nextLine Hatası
- 16. Kendini Test Eden Sınıf: Car
- 17. Metot Parametreleri: Değerle Aktarım
- 18. Dizileri Metoda Göndermek
- 19. Karar ve Döngü Yapıları
- 20. for-each Döngüsü
- 21. String ile Swap ve StrClass
- 22. Nesne Dizisinde for-each: Kütüphane Örneği
- 23. toString Metodunu Ezmek

1. Bir Düzeltme ve Önceki Derslerin Özeti

Hoca derse, kendi ders kaydını dinlerken fark ettiği bir kelime hatasını düzelterek başladı. UML sıralama diyagramı örneğinde bir `client` vardı. Bu `client`, `Car` sınıfından üretilmiş `kit` nesnesinin `getPlate` metodunu çağırıyordu. Hoca bu çağrıya önceki derste "response" demişti. Oysa bu çağrı bir istektir (request). Bu isteğe gelen karşılığa ise cevap (reply ya da response) denir. Yani `client`, `kit` nesnesine bir mesaj gönderir, bir istekte bulunur, ardından ona cevap gelir.

Ardından hoca önceki derslerin kısa bir özetini yaptı. Sınıf, verilerden ve metotlardan oluşur. Sınıf yapısı soyuttur, ondan üretilen nesne ise somuttur. "Araba" dediğimiz şey soyuttur, ondan üretilen belirli, fiziksel bir araba somuttur. Boş bir sınıf yazmak aslında çok kısadır. Ona

problemin türüne göre anlam yükleyip özellikler ve fonksiyonlar atamak ise nesneye yönelik tasarım gerektirir.

Bir sınıfın alanları ilkel (primitive) ya da ilkel olmayan (non-primitive) olabilir. Daha önce yapılan örnekte kitabın içinde bir yazar tutulmuş, sonra bu kitaplar bir kütüphanede toplanmıştı. Hoca burada kavramlara dikkat çekti: kitap başka şeydir, kitapların dizisi başka şeydir.

> **Hocanın vurgusu:** Terminoloji önemlidir. Veri için member, field, attribute, property gibi kelimeler kullanılır. Fonksiyonlar için member method, function, method, action, task gibi kelimeler kullanılır.

UML gösterimleri de tekrar edildi. Sınıf diyagramı sınıfın görünümünü, nesne diyagramı ise sınıftan üretilen bir nesnenin görünümünü verir. Bir diyagramda kutunun nesne olduğunu, adının altının çizili olmasından anlarız. Örneğin `client` bir nesnedir, `kit` de `Car` sınıfından üretilmiş bir nesnedir. Hoca etkileşim diyagramlarından sıralama (sequence) ve iş birliği (collaboration) diyagramlarını andı. Bunların slaytlardaki düzene göre yeri geldikçe anlatılacağını, bir kısmının Sistem Analizi ve Tasarımı dersinde, bir kısmının da yazılım mühendisliğinde görüleceğini söyledi. Çünkü yazılımla ilgili hemen her şeyin UML ile bir bağlantısı vardır.

> **Ders sonrası not:** Hoca rolleri atayan diyagramı anarken "Activity diagram mıydı?" diye sordu ve soru yanıtsız kaldı. Altyazıda doğrulanmıyor; roller için kullanılan diyagram büyük olasılıkla kullanım durumu (use case) diyagramıdır.
>
> **Hocanın vurgusu:** UML gösterimleri çok önemlidir. Hoca, UML verilip kodunun sorulabileceğini söyledi.

Son olarak nesnelerin referans tipli, ilkel tiplerin değer tipli olduğu ve bellekte farklı yerlerde saklandıkları hatırlatıldı.

2. Paketler ve import

Konuya bir öğrenci sorusuyla geçildi. Slaytta `paket1.*` biçiminde bir import vardı ve alt paketteki sınıfın bununla alınmadığı yazıyordu.

Soru: Slaytta alt paketteki sınıfın import edilmediği yazıyor. Neden edilmiyor?

Cevap: Bir paket adı verip yıldız yazdığınızda yalnız o paketin içindeki sınıf dosyaları eklenir. Paketin altındaki paketler dahil edilmez. Altında bir paket daha varsa onu da ayrıca import etmek gerekir.

Hoca bunu NetBeans ortamında iç içe paketler açarak gösterdi. Aşağıdaki küçük örnek aynı durumu özetliyor. `paket1` içinde `A`, `paket1.paket2` içinde `C` sınıfı olsun.

```
// Dosya: paket1/A.java
package paket1;
public class A { }

// Dosya: paket1/paket2/C.java
package paket1.paket2;
public class C { }

// Dosya: Dogru.java
import paket1.*;
import paket1.paket2.*;
public class Dogru { A a = new A(); C c = new C(); }
```

Javac ile denendi: ikinci import satırı silinince derleyici `C` için `cannot find symbol` hatası veriyor. İki satır birlikte yazılınca kod derleniyor.

2.1 C'deki #include Benzetmesi

Hoca bunun nedenini C'deki `#include` ile anlattı. C'de `test.c` dosyasından derleme yapılırken önce önişlenmiş `test.i` dosyası oluşur. Dosyanın içinde yalnızca basit bir toplama işlemi, bir `#include`, bir makro tanımı ve yorumlar vardı. Ama `#include` eklendikten sonra 3-5 satırlık kod neredeyse 600 küsur satıra çıkıyordu. Derleyici bunlardan ilgili olanları alıp gereksizleri atıyordu. Hoca, `import` için de aynı mantığı kurdu: bütün dosyalar dahil edilseydi program çok

büyürdü, yıldızın alt paketleri almaması gereksiz dosya eklenmesini engellemek içindir. Örnek olarak `Scanner` sınıfının bulunduğu paketi verdi. Orada iç içe birçok paket ve sınıf vardır.

> **Ders sonrası not:** Hoca "`import` etmek kodu `#include` gibi programa dahil eder, program büyür" dedi; doğrusu bu benzetmenin Java'da geçerli olmadığıdır. `import` yalnız sınıf adlarının kısa yazılmasını sağlar. Javac ile denendi: dört paket yıldızla import eden sınıf ile hiç import içermeyen aynı sınıfın `.class` dosyaları, satır numaraları eşitlendiğinde bayt bayt aynı çıktı. Bu yüzden gereksiz import programı büyütmez. Ama hangi sınıfın nereden geldiğini görmek için yalnız gereken sınıfı yazmak yine okunaklıdır.

2.2 Paket Hiyerarşisinin Anlamı

Soru: Alt paketin adını neden gösteriyoruz? `paket1`'e girmeden `paket2`'ye ulaşamıyor muyuz?

Cevap: Ulaşılır. `paket1`'dekileri almadan yalnız `paket2`'nin altındakileri alabilirsiniz. Üsttekini eklemeye gerek yoktur. Hiyerarşi, aralarında mantıksal ilişki olan parçaları, yani programın modüllerini ayırmak içindir. Modülerliği artırmaya yarayan mantıksal bir düzenlemedir.

Javac ile denendi: yalnız `import paket1.paket2.*;` yazılınca `C` sınıfı sorunsuz kullanılıyor.

Hoca ileride grafik arayüz ve olay işleme (event handling) konularında bunu göreceğimizi söyledi. Önce AWT paketi eklenecek, o yetmeyecek, sonra altındaki olay paketi ayrıca eklenecek. Tek bir sınıf gerekiyorsa da doğrudan o sınıf import edilir.

3. Bilgi Gizleme: Üyeleri Neden Gizliyoruz?

Sonraki konu görünürlük kuralları ve bilgi gizlemeydi (information hiding). Hoca akla gelebilecek şu soruyu kendisi sordu: Sınıfın üyelerini neden gizliyoruz? Zaten onlara yapıcı (constructor) ya da set/get metotlarıyla erişiyoruz.

Cevap, bu erişimlerin normal erişimden farklı olmasıdır. Yapıcı ile erişim bir defalıktır ve yapıcı nasıl tanımlandıysa ona göre olur. Bir değişkeni `public` yaparsak ise her defasında erişip değiştirebiliriz. Bu, nesneyi korumaya ve sınıfların birbiriyle uyumlu çalışmasına aykırıdır.

> **Hocanın vurgusu:** Bu yüzden üyeler genellikle `private` yapılır. Hoca, size sorulan sorularda bütün üyeleri `private` yapmanızın bekleneceğini söyledi.

Şasi numarası örneği bunu güzel gösterir. Araba üretilir üretilmez ona bir numara verilir. Kitap üretilir üretilmez ona bir ISBN verilir. Bu numara daha sonra değişmez, bu yüzden alan yapıcı ile atanır. Fiyat gibi sonradan değişebilecek alanlar ise yine doğrudan değil, set ve get metotlarıyla değiştirilir.

Bir başka sebep de kodun başkaları tarafından kullanılmasıdır. Yazdığınız koddan kalıtım alınabilir ve sınıfın davranışı değiştirilebilir. Kullanma ilişkisiyle de elde edilip içeriği değiştirilebilir. Kodun içeriği değişince projenin geri kalanının çalışması aksayabilir.

Hocanın günlük hayattan benzetmesi telefondu. Telefonun içinde bir sürü devre vardır. Onları görüp değiştiremeyiz, yalnız bize lazım olan kısımlara erişiriz. Yazılım daha "yumuşak" bir şey gibi görünse de işin ucunda banka olabilir, veri gizliliği ve güvenliği gerektiren durumlar olabilir. Her zaman her şeye erişebilmek doğru değildir.

4. UML'de Erişim Sembolleri ve Yazım Düzeni

UML'de erişim belirteçleri sembollerle gösterilir:

- `+` : `public`
- `-` : `private`
- `#` : kalıtımla ilgili erişim (`protected`, daha sonra görülecek)
- `~` : paketle ilgili erişim (daha sonra görülecek)

> **Hocanın vurgusu:** Bunu iyi bilmeniz lazım. Bir kod verilip UML'i çizilmesi istenirse şu düzene uyulur.

1. Önce erişim belirteci (`public`, `private` karşılığı sembol) yazılır.

2. Sonra özelliğin ya da metodun adı yazılır.
3. Özellikse iki nokta konur ve türü yazılır. Bu tür `int`, `float` ya da `Kitap`, `Kalem` gibi bir sınıf olabilir.
4. Metotsa iki nokta konur ve dönüş tipi yazılır.
5. Parametre eklenecekse yine önce parametrenin adı, iki nokta, sonra parametrenin tipi yazılır.

Hoca bunu bir akış şeması gibi uyulması gereken bir düzen olarak tanımladı. Kullanılan UML yazılımları da buna uygun tasarlanmıştır.

5. final ve static Tekrarı

`final` bir alanın değeri bir kez verildikten sonra değiştirilemez. `final` bir metot ise daha sonra kalıtımla ezilemez (override edilemez).

`static` üyelerde ilk değer verme (initialization) bir defalıktır. Sonrasında yalnız atama (assignment) yapılabilir. C'de olduğu gibi, bir kez ilk değer verilir, sonra sürekli ++ yapılabilir. `static` üye, nesnenin değil bütün sınıfın özelliğidir.

> **Ders sonrası not:** Hoca "Bir metot static ise bir defaya mahsus initialization yapılıyor" dedi; doğrusu bu cümle `static` alan içindir. Hemen ardından verdiği sayaç örneği de bir alan örneğidir.

Hocanın önceki derste verdiği örnek kantin örneğiydi. Kantine kaç öğrencinin girdiğini herkesin görebildiği ortak bir alanda tutarız. Aynı şekilde bir arabaya ait parça sayısı da böyle düşünülebilir. Bu ortak alan olması gereken en yüksek değere ulaşınca artık yeni müşteri kabul etmeyiz. `static` üyeler genellikle nesnelerin hep ortak olan özelliklerini belirlemek için kullanılır ve sınıfın değişkenleridir.

Üyeler ve metotlar `static` ise sınıfın adıyla erişilir, örneğin `Kitaplar.kitapTanit()` gibi. Nesne üzerinden de erişilebilir, ama bu her zaman tercih edilen bir yol değildir. `static` olmayan bir metoda ise sınıf üzerinden erişilemez. Önce nesne üretilir, sonra o nesnenin metodu çağrılır. Çünkü elimizdeki soyut şeyi önce somut hale getirip o somut hal üzerinden çalışmamız gerekir.

Yapıcıların aşırı yüklenebildiği de hatırlatıldı. Yapıcıların devamına kalıtım konusunda, aşırı yüklemeye de metotlar konusunda geri dönülecek. Hoca o noktada C# ya da C++ ile operatörlerin aşırı yüklenmesine de bakılacağını söyledi.

6. Tam Sayı Tipleri ve Tür Dönüşümü

Bu noktadan sonra hoca slaytlara geçti ve slayt kısımlarını çoğunlukla İngilizce anlattı. İlkel tipler yapısal programlama dersinden bilinir. Tek bir birim değer tutarlar. Tam sayı değerleri için dört tip vardır. Aralarındaki fark boyutlarıdır:

Tip	Boyut (bayt)
`byte`	1
`short`	2
`int`	4
`long`	8

Javac ile denendi: `Byte.BYTES`, `Short.BYTES`, `Integer.BYTES` ve `Long.BYTES` sırasıyla 1, 2, 4 ve 8 veriyor.

Boyut, tipler arası dönüşümde önemlidir. Küçük boyuttan büyüğe dönüşüm serbesttir. Tersine Java derleyicisi izin vermez. Hoca bunu derleyerek gösterdi: `int`'ten `long`'a atama çalıştı, tersi derleme hatası verdi. Aynı şekilde `byte`'tan `short`'a atama serbest, tersi değildir.

```java
// Dosya: Donusum.java
public class Donusum {
    public static void main(String[] args) {
        int a = 5;
        long b = a;          // küçükten büyüğe: serbest
```

```
        byte c = 5;
        short d = c;          // küçükten büyüğe: serbest
        System.out.println(b + " " + d);
    }
}
```

Javac ile denendi: kod derleniyor ve `5 5` yazıyor. Ters yöndeki `int a = b;` satırı şu hatayı veriyor: `incompatible types: possible lossy conversion from long to int`. Derste de aynı hata gösterildi. `byte c = d;` satırı da aynı türde hata veriyor (`from short to byte`).

7. İlk Swap Denemesi: Integer ile Çalışmıyor

Bütün ilkel tiplerin, bazı metotları ve faydalı işlemleri sağlayan bir sarmalayıcı sınıfı (wrapper class) vardır. Hoca bunun bir örneği olarak iki sayıyı yer değiştirmeyi (swap) aldı. C'de iki tam sayıyı bir fonksiyon içinde yer değiştirmek için işaretçi kullanılır. Hoca slaytı anlatırken sarmalayıcı sınıfların bu işi kolaylaştıracağını söyledi ve denemeye geçti.

> **Ders sonrası not:** Hoca burada üç şey söyledi: "C'de call by value yoktu", "Java'da ilkellerde call by value yok", "wrapper class'ın call by reference özelliği var". Üçü de yanlıştır ve hoca dersin ilerleyen kısmında kendisi düzeltti. C'de ve Java'da parametreler varsayılan olarak değerle aktarılır. Sarmalayıcı sınıflar referansla aktarım sağlamaz. Aşağıdaki deneme de bunu gösteriyor.

Önce `int` parametreli bir `swap` yazıldı. Sonuç beklendiği gibiydi: Değerle aktarım yüzünden metot içindeki değişiklik `main`'e yansımadı. Ardından aynı metot `Integer` ile yazıldı.

```
// Dosya: SwapDeneme.java
public class SwapDeneme {
    static void swap(Integer x, Integer y) {
        Integer t = x;
        x = y;
        y = t;   // yalnız swap'in yerel referansları yer değiştirir
    }

    public static void main(String[] args) {
        Integer x = 3, y = 6;
        System.out.println("before: " + x + " " + y);
        swap(x, y);
        System.out.println("after:  " + x + " " + y);
    }
}
```

Derste sonuç `3 6 3 6` oldu, yani yer değişmedi. Javac ile denendi, aynı sonuç çıkıyor. Hoca `new Integer(3)` biçimini de denedi, yine değişmedi. Bu noktada şunu söyledi: Burada da referanslar değerle kopyalanıyor, yani `swap` içinde kendi referansları yer değiştiriyor. Konuyu dersin ilerleyen kısmına bıraktı.

> **Ek bilgi (derste anlatılmadı):** `Integer x = 3;` yazımında `int` değer derleyici tarafından otomatik olarak `Integer` nesnesine sarılır. Buna otomatik kutulama (autoboxing) denir. Bu yüzden `new` yazmadan `Integer` değişkenine sayı atanabiliyor. Javac ile denendi: bu satır uyarısız derleniyor.

Soru: `x = y` yerine `x.intValue() = y.intValue()` desek olmaz mı?

Cevap: Olmaz. `intValue` bir set metodu değildir, parametre almaz, bir get metodudur.

Javac ile denendi: bu satır `unexpected type, required: variable, found: value` hatası veriyor.

Soru: Peki `intValue`, `x`'in bir özelliği olabilir mi, `x.value` gibi?

Cevap: Hayır. Parantezle çağrıldığı için `intValue` bir metottur. Bir özellik olsaydı böyle çağrılmazdı.

> **Ders sonrası not:** Hoca bu cevapta `Integer`'ın "herhangi bir özelliği yok" dedi; doğrusu `Integer` sınıfının içinde `value` adlı bir alan vardır, ama `private final` tanımlıdır.

Javac ile denendi: `x.value` yazınca `value has private access in Integer` hatası alınıyor. Sınıfı yansıtma (reflection) ile incelemek de alanın `private final int value` olduğunu gösteriyor. Hocanın sarmalayıcıyı tanımlarken söylediği "içinde tek üye vardır" cümlesi de bununla uyumludur.

8. Diğer İlkel Tipler ve İlkel Veri Üzerinde İşlemler

Bütün tam sayı tiplerinin sarmalayıcısı vardır: `Integer`, `Long`, `Short`, `Byte`. Kayan noktalı tipler C'deki gibi `double` ve `float`'tır, sarmalayıcıları `Double` ve `Float`'tır. Bütün sarmalayıcı sınıf nesneleri bellekte heap bölgesinde tutulur. `char` tipinin sarmalayıcısı `Character`, `boolean` tipininki `Boolean`'dır. Java'da karakterler için UTF-16 kodlaması kullanılır. Javac ile denendi: `Character.BYTES` 2 veriyor. `String` ise ilkel olmayan bir tiptir ve ayrıca ele alınacak.

İlkel veriler üzerindeki işlemler C ile aynıdır. Hoca bu yüzden üzerinde durmadı:

- Aritmetik işlemler: toplama, çıkarma, mod alma, çarpma.
- `++` ve `--`: ön ve son artırma/azaltma. İşlem önceliği (operator precedence) C'deki gibi önemlidir. Hoca isteyene tablo paylaşabileceğini söyledi.
- Matematiksel işlemler: `pow`, `abs`, `round` gibi.
- Bit düzeyinde işlemler: değil (~), ve (&), veya (|), özel veya (^) ve kaydırma işlemleri.

Slayttaki bit örneği, bir `n` sayısının sağdan dördüncü bitini almaktı: `(n & 8) / 8`. 8 sayısının ikili yazılışı `1000` olduğu için (ara adım eklendi) `n & 8` yalnız dördüncü biti bırakır. Sonuç 8 ya da 0 olur. 8'e bölünce 1 ya da 0 kalır. Python ile 0-4095 aralığındaki bütün sayılar için hesaplandı: bu ifade `(n >> 3) & 1` ile aynı sonucu veriyor.

9. Sarmalayıcı Sınıflar

Sarmalayıcı sınıf, içinde bir ilkel tip barındıran tek üyeli özel bir sınıftır. İlkel değeri sarar ya da kutular (wrap, box). Örneğin `Integer` bir sınıftır ve içinde tek bir üye vardır. Bu sınıf üzerinden başka fonksiyonlar ve işlemler tanımlanır.

Sarmalayıcılar iki sebeple kullanılır: faydalı metotları için ve ilkel tipin kullanılamadığı durumlar için. Hoca ikinci duruma örnek olarak serileştirmeyi (serialization) ve eşleme indeksini (map index) gösterdi. Bunlar ileride akışlar ve koleksiyonlar bölümlerinde görülecek. Sarmalayıcılar `java.lang` paketindedir.

> **Ders sonrası not:** Hoca slaytı anlatırken ilkel tipin kullanılamadığı durumlara swap işlemini de ekledi; doğrusu sarmalayıcı sınıf swap sorununu çözmez. Bölüm 7'deki deneme ve hocanın sonraki açıklaması bunu gösteriyor.
>
> **Ek bilgi (derste anlatılmadı):** `java.lang` paketi her Java dosyasında otomatik olarak görünür. Bu yüzden `Integer`, `String`, `Math` ve `System` için `import` yazılmaz. Javac ile denendi: bu sınıflar import satırı olmadan derleniyor, `java.util.Random` ise import olmadan `cannot find symbol` hatası veriyor.

9.1 Integer'ın Faydalı Metotları

Hocanın saydığı metotlar şunlardır:

- `compareTo`: iki tam sayıyı karşılaştırır.
- `intValue`: nesnenin içindeki `int` değeri verir.
- `static int parseInt(String)`: verilen metni tam sayıya çevirir.
- `toString` ve `static toString`: bunlar farklı metotlardır. Birincisi nesne üzerinden çalışır, ikincisi `static`'tir.
- `static Integer valueOf(String)`: metinden ya da sayıdan `Integer` nesnesi üretir.

`parseInt`, argümanı tam sayıya çevirmeye çalışır. Mümkünse sonucu verir, mümkün değilse bir istisna (exception) fırlatır. Hoca bunu sayıya çevrilemeyen `"BDDSF"` metniyle denedi.

```java
// Dosya: IntegerMetotlari.java
public class IntegerMetotlari {
    public static void main(String[] args) {
        Integer sarma1 = Integer.valueOf(1);
```

```
        Integer sarma2 = Integer.valueOf(7);
        System.out.println(sarma1.compareTo(sarma2));   // -1

        Integer t = Integer.parseInt("BDDSF");          // burada istisna fırlar
        System.out.println(t);                          // bu satır çalışmaz
    }
}
```

Javac ile denendi: Çıktının ilk satırı `-1`, ardından `NumberFormatException: For input string: "BDDSF"` geliyor. Hocanın gösterdiği gibi istisnadan önceki satır çalışır, sonraki çalışmaz.

`compareTo` sonucu burada `-1`'dir, çünkü argüman olan `sarma2` (7), `sarma1`'den (1) büyüktür.

Hoca `new Integer(...)` kullanımının da mümkün olduğunu, ama Oracle tarafından eskitildiğini (deprecated) söyledi. Javac ile denendi: JDK 25'te `new Integer(3)` derleniyor, ama `Integer(int) ... has been deprecated` uyarısı veriyor.

10. String Sınıfı

`String`, önemli bir veri tipidir ve aynı zamanda bir sınıftır. Hocanın saydığı karşılaştırma metotları şöyledir: `compareTo` büyük/küçük harfe duyarlıdır, `compareToIgnoreCase` duyarlı değildir. `System.out.println` bir metin argümanı alır. `print` ile arasındaki küçük farka çıktı konusunda dönülecek.

10.1 compareTo Nasıl Karşılaştırır?

Slaytta iki metne sabit atama (literal) yapılıyordu. `compareTo` sonucu negatif çıkıyor, bu da `b`'nin `a`'dan büyük olduğunu gösteriyordu. Hoca yöntemi şöyle anlattı: metot iki metni karakter karakter karşılaştırır, `karakterA - karakterB` çıkarmasını yapar ve bu sonucu döndürür.

> **Ders sonrası not:** Hoca önce sonucun "sıfır, artı bir ya da eksi bir" olduğunu söyledi; doğrusu `String.compareTo` için sonuç, kendi anlattığı gibi, karakter farkıdır. Yani herhangi bir tam sayı olabilir ve önemli olan işaretidir. `-1`, `0`, `1` sonucu bu örneklerde `Integer.compareTo` için geçerlidir.

10.2 String Sabit Havuzu

Ardından hoca bir soru sordu: `aa` ve `bb` değişkenlerinin ikisine de `"hello"` atanırsa `compareTo` ne döndürür? Cevap sıfırdır. Hoca bunu, hakkında iyi bir açıklama görmediği, istisna gibi bir durum olarak sundu ve şöyle açıkladı.

Çift tırnakla her string sabiti oluşturulduğunda Java sanal makinesi string sabit havuzunu (string constant pool) kontrol eder. Bu metin daha önce atanmışsa ikinci metin için yeni bellek ayrılmaz. İki değişken de aynı bellek konumunu gösterir. Java bu işi tekrar yapmamak için metni saklar. Hoca bunun Java'nın bir özelliği olduğunu, başka dillerde belki böyle olmayabileceğini söyledi.

> **Ders sonrası not:** Hoca `compareTo` sonucunun sıfır çıkmasını havuza, yani değişkenlerin aynı yeri göstermesine bağladı; doğrusu sıfır, içerikler karakter karakter aynı olduğu için çıkar. Bu bir istisna da değildir. Havuz yalnız iki değişkenin aynı nesneyi göstermesini açıklar. Javac ile denendi: `String cc = new String("hello");` ile üretilen metin için `aa == cc` sonucu `false`, ama `aa.compareTo(cc)` sonucu yine `0` ve `aa.equals(cc)` sonucu `true`. `aa == bb` sonucu ise iki sabit için `true`. Metin içeriği `equals` ya da `compareTo` ile karşılaştırılmalıdır.

Hoca farkı göstermek için kısa bir karşılaştırma daha yaptı ve ekranda `false` aldı. Nesneler farklı olduğu için böyle olduğunu, bunu geçen hafta kitap örneğinde de gördüğümüzü hatırlattı.

10.3 Karakter Dizisinden String'e

Bir karakter dizisi doğrudan `String`'e çevrilebilir. `String` sınıfının yapıcılarına bakıldığında `byte` dizisi, `char` dizisi gibi argümanlar alan birçok aşırı yüklenmiş sürüm olduğu görülür. Ters

dönüşüm, yani metinden karakter dizisine geçiş `toCharArray` ile yapılır.

10.4 Faydalı Metotlar

Hoca metotları anlamlarını öğrencilere tahmin ettirerek anlattı. Aşağıdaki kod hocanın değindiği örnekleri bir araya getiriyor. `ss` değişkeninin değeri örnek amaçlı seçildi.

```
// Dosya: StringDeneme.java
public class StringDeneme {
    public static void main(String[] args) {
        String aa = "hello";
        String bb = "hello";
        System.out.println(aa.compareTo(bb));              // 0

        char[] ch = {'J', 'a', 'v', 'a'};
        String s = new String(ch);
        System.out.println(s);                             // Java

        String ss = "ders";
        System.out.println(ss.charAt(0));                  // d
        System.out.println(ss.charAt(ss.length() - 1));    // s

        String newSs = ss.concat(" hello");
        System.out.println(ss);                            // ders
        System.out.println(newSs);                         // ders hello

        System.out.println("relationship".endsWith("ship"));   // true
        System.out.println("unhappy".substring(2));            // happy
        System.out.println("hamburger".substring(4, 8));       // urge
        System.out.println("İstanbul".contains("Yıldız"));     // false
    }
}
```

Javac ile denendi: yorumlarda yazan sonuçların hepsi çıkıyor. Metotlar tek tek şöyledir:

- `charAt(i)`: verilen konumdaki karakteri verir. İlk karakterin konumu 0'dır. Dizilerdeki gibi çalışır.
- `length()`: metnin uzunluğunu verir. Son karakterin konumu `length() - 1`'dir.
- `concat(metin)`: iki metni birleştirir. Dönüş tipi `String`'tir. Dönen değer alınmalıdır, çünkü `ss`'nin sonuna bir ekleme olmaz, bu yeni bir metindir.
- `contains(metin)`: dönüş tipi `boolean`'dır. Metnin içinde verilen alt metnin olup olmadığını denetler. Bunu derste bir öğrenci tahmin etti, hoca onayladı.
- `contentEquals`: metni belirtilen karakter dizisiyle karşılaştırır. Aynı karakterlerden oluşuyorsa `true` döner. `StringBuffer` ve `CharSequence` ile çalışan sürümleri vardır.
- `endsWith(sonek)`: metin verilen sonekle mi bitiyor, onu denetler. Hoca bunun doğal dil işlemede çok önemli olduğunu söyledi.
- `equals`, `equalsIgnoreCase`: metni verilen nesneyle karşılaştırır. İkincisi büyük/küçük harfe duyarsızdır.
- `getBytes`: metni bayt dizisine çevirir.
- `getClass`, `hashCode`: daha sonra görülecek.
- `indexOf`: aranan karakterin ya da alt metnin ilk geçtiği konumu verir. Hoca dört aşırı yüklü sürümünü saydı: karakter arama, belirtilen konumdan itibaren karakter arama, alt metin arama, belirtilen konumdan itibaren alt metin arama.
- `lastIndexOf`: aynı aramayı son geçtiği konum için yapar.
- `isEmpty`: metnin boş olup olmadığını söyler.
- `split`: metni bir düzenli ifadeye (regular expression) göre parçalara böler.
- `substring(baslangic)`: verilen konumdan sona kadar olan parçayı verir. `"unhappy"` için 2. konum `h`'dir, sonuç `"happy"`'dir.
- `substring(baslangic, bitis)`: son konum dahil değildir. `"hamburger".substring(4, 8)` u'dan e'ye kadar olan `"urge"` metnini verir, 8. konumdaki `r` dahil değildir.
- `toCharArray`: metni karakter dizisine çevirir.

- `toLowerCase`, `toUpperCase`: küçük/büyük harfe çevirir. Karakter zaten büyükse olduğu gibi kalır.
- `trim`: boşlukları temizler (aşağıya bakın).

  Ders sonrası not: Hocanın saydığı dört `indexOf` sürümü slayttaki sürümdür. Javac ile denendi: JDK 25'te yansıtma ile 6 sürüm görünüyor, ek ikisi başlangıç ve bitiş konumu alıyor.

10.5 trim ve Arama Kutusu Örneği

Hocanın `trim` için örneği şuydu: Arama kutusuna kitap adı girilecek. Kullanıcı yalnız boşluk girerse veritabanında arama yapılırken bir sorun çıkabilir. Bunu önlemek için önce `trim` çağrılır. Bazı web sayfalarında hiçbir şey yazmayıp boşluk girdiğinizde sayfa bunu kabul eder, bazılarında etmez. Etmeyenlerde boşluklara göre `trim` yapılmaktadır.

> **Ders sonrası not:** Hoca "`trim` boşluk dışındaki karakterleri döndürür, boşlukları yok eder" dedi; doğrusu `trim` yalnız metnin başındaki ve sonundaki boşlukları siler, aradaki boşluklara dokunmaz. Javac ile denendi: `"  a b  ".trim()` sonucu `"a b"`, yalnız boşluktan oluşan metinde sonuç boş metindir (`isEmpty()` `true`).

10.6 Slayttaki Örnek

Slaytta `String a` için `İstanbul`, `String b` için `Yıldız` atanmıştı. `a.contains(b)` sonucu `false`'tur, çünkü içerme için metnin birebir geçmesi gerekir. Javac ile denendi: sonuç `false`. Hoca bir sonraki satır için `true` dedi. `toUpperCase` karakterleri büyüğe çevirir, zaten büyük olan karakter aynı kalır. Sonraki satırlar yine `toUpperCase` ve `contains` denemeleriydi.

10.7 Metin İşleme ve Kaynaklar

Hoca metin işlemenin bir dönemi alacak kadar geniş olduğunu, doğal dil işlemeye kadar gittiğini söyledi. Çünkü bir metni alıp işleyip ondan anlam çıkarmak metin işlemleri içindedir. Java düzenli ifadeleri de bu konuya dahildir. Doğal dil işleme gibi bir proje yapacaksanız önce metne ön işlem (preprocessing) yaparsınız: boşlukları silmek, karakter sayısını bulmak, bazı sonekleri almak ya da atmak gibi. Örneğin `relationship` sözcüğünde `ship` bir sonektir. `ship` aranıp atılırsa `relation` elde edilir. Bir türden başka bir türe dönüşüm yapmak için de sonekler kullanılır.

Bu ders kapsamında metin üzerinde çok fazla işlem yapılmayacak, temel işlemleri bilmek yeterlidir. Hoca ek kaynak olarak JavaTpoint'in String belgelerini ve W3Schools sitesini önerdi.

11. Kendi Sarmalayıcı Sınıfımız: Swap'i Çalıştırmak

Aradan sonra hoca, başta çalışmayan swap işine geri döndü. Hazır sarmalayıcı sınıflarda referansla aktarım çalışmıyordu. Bu kez `MyClass` adında bir sınıf yazdı ve içine yalnız bir tam sayı alan ekledi. Alanı `private` yaptı. NetBeans'te Refactor ve Encapsulate komutlarıyla set ve get metotlarını ürettirdi. Bir de yapıcı yazdı. Ardından `MyClass` türünden 4 ve 8 değerli iki nesne üretti. Artık metoda gönderilen şeyler bir değişken değil, nesnelerdi.

```
// Dosya: SwapTest.java
class MyClass {
    private int a;

    public MyClass(int a) {
        this.a = a;
    }

    public int getA() {
        return a;
    }

    public void setA(int a) {
```

```
        this.a = a;
    }
}

public class SwapTest {
    // Ad aynı, parametre türü farklı: aşırı yükleme
    static void swap(Integer x, Integer y) {
        Integer t = x;
        x = y;
        y = t;
    }

    static void swap(MyClass x, MyClass y) {
        int temp = x.getA();
        x.setA(y.getA());
        y.setA(temp);
    }

    public static void main(String[] args) {
        MyClass a = new MyClass(4);
        MyClass b = new MyClass(8);
        System.out.println("before: " + a.getA() + " " + b.getA());
        swap(a, b);
        System.out.println("after:  " + a.getA() + " " + b.getA());
    }
}
```

Derste sonuç “4, 8, 8, 4” oldu. Javac ile denendi, aynı sonuç çıkıyor: önce `4 8`, sonra `8 4`. Hoca şöyle özetledi: Aslında `Integer` için kendim bir sarmalayıcı sınıf yazdım. Yani swap mümkün değil diyemeyiz, ama hazır sarmalayıcı sınıflarda doğrudan yok. Alanlar `public` olsaydı set ve get yerine doğrudan atama da kullanılabilirdi.

11.1 Aşırı Yükleme ve Belirsizlik

Aynı adda birkaç `swap` yazılmasına rağmen derleyici hata vermedi. Hoca bunu aşırı yükleme (overloading) olarak açıkladı. Parametreleri `long` ve `short` yaparak ne olacağını da denedi. Derleyici sayıları doğrudan `int` olarak algılıyor, kafası karışmıyordu. Javac ile denendi: `int`, `long` ve `Integer` parametreli sürümler bir arada iken `s(3, 5)` çağrısı `int` sürümüne, iki `Integer` değişkenle yapılan çağrı `Integer` sürümüne gidiyor.

Hoca asıl riski şöyle belirtti: İki metot arasında derleyicinin kafası karışırsa hangisini çağıracağını bilemez ve derleme zamanı hatası alırız. Buna belirsizlik (ambiguity) denir. Önemli olan, hangi aşırı yüklemeye gidileceği bilinecek şekilde tanım yapmaktır. Örneğine metot aşırı yükleme konusunda bakılacak.

11.2 Neden Integer ile Olmadı, MyClass ile Oldu?

Hoca bellek şemasını çizerek anlattı. Şema adım adım şöyle okunur:

```
main içinde                        bellekte
  Integer x  --> adres 100 -->     [ 4 ]
  Integer y  --> adres 110 -->     [ 8 ]

swap(x, y) çağrılınca: adresler kopyalanır
  swap'in x'i = 100
  swap'in y'si = 110

swap içinde x ile y yer değiştirirse:
  swap'in x'i = 110
  swap'in y'si = 100
  main'deki x = 100, main'deki y = 110 (değişmedi)
```

1. `main` içinde iki `Integer` nesnesi vardır. Bellekte bir yerlerde saklanırlar ve içlerinde 4 ve 8 gibi değerler tutarlar. Hocanın varsaydığı adresler 100 ve 110'dur.
2. `swap` içine gelindiğinde `swap`'e ait yeni `x` ve `y` değişkenleri tanımlanır. Adları aynı da olabilir.
3. Bunlar referans tuttuğu için 100 ve 110 adresleri bu yeni değişkenlere kopyalanır. `swap` için yeni bellek hücreleri ayrılmıştır.
4. İçeride yapılan yer değiştirme burada kalır, `main`'e yansımaz. 4 ve 8'in bulunduğu yerin kendisine ulaşılabilseydi değiştirilebilirdi. Ama `Integer` buna izin vermez.

Hoca bunu referansın değerle aktarılması olarak adlandırdı ve buna "yalancı call by reference" dedi. Çağrılan referans kopyalanır, işlemler kopya üzerinde yapılır ve yapılan orada kalır. Hoca bunun C'de de olmadığını ekledi.

`MyClass` örneğinde de `x` ve `y` yine kopyalanmış referanslardır. Fark şudur: Bu kez referanslar yer değiştirmiyor. Kopya referans da aynı nesneyi gösterdiği için `setA` ile nesnenin içi değiştiriliyor. Bu değişikliği çağıran taraf da görüyor. Hoca `Integer` ile yapılan denemede ise set ve get ile erişilmediğini, doğrudan birbirine atanmaya çalışıldığını belirtti.

> **Ders sonrası not:** Hoca kendi sınıfımızı yazmayı "call by reference haline getirmek" diye tanımladı; doğrusu burada da aktarım değerledir. `SwapTest` örneğinde referansların kopyası gider. Değişen şey referans değil, gösterilen nesnenin içeriğidir. Hocanın "yalancı" nitelemesi ve Bölüm 17'deki açıklaması da bununla uyumludur.

12. Math Sınıfı

Şimdiye kadar `String` ve sarmalayıcı sınıflar görüldü. `Math` ve `Random` ise matematiksel işlemlere özel sınıflardır. `Math.random()` metodu 0.0 ile 1.0 arasında `double` bir değer üretir. Argüman verilmez.

> **Ders sonrası not:** Hoca slaytı okurken bu metodun birkaç aşırı yüklü sürümü olduğunu söyledi, sonra parametreli sürümü denedi ve derleyicinin kabul etmediğini gördü; doğrusu `Math.random` tek sürümlüdür ve parametre almaz. Javac ile denendi: `Math.random(5)` için `required: no arguments` hatası çıkıyor.

Hoca sordu: Bu metoda nesne üretmeden nasıl erişebildik? Öğrencilerden gelen cevap doğruydu: Metot `static`'tir. `contains` metoduna nesne üzerinden erişiyoruz, ama `Math.random`'ı doğrudan kullanabiliyoruz.

`Math` sınıfının içinde sık kullanılan matematik fonksiyonları için birçok `static` metot vardır. Tanımları `public static`, dönüş tipi ve parametreleri çoğunlukla ilkel tiptedir.

- `abs`: mutlak değer.
- `max`, `min`: iki sayıdan büyüğü, küçüğü.
- `ceil`: yukarı yuvarlar.
- `floor`: aşağı yuvarlar.
- `round`: yuvarlama kuralına göre yuvarlar. Ondalık kısım 5 ya da daha büyükse soldaki basamak bir artar.
- `sqrt`: karekök. Hoca, karekökün hesaplanması için bir algoritma gerektiğini hatırlattı.
- `pow` ve üstel işlemler de bu sınıftadır.

13. Sözde Rastgele Sayılar ve Random Sınıfı

Daha fazla rastgele işlem için `Random` sınıfı kullanılır. Bu sınıf sözde rastgele (pseudo-random) sayı üretir.

Soru: Sözde rastgele sayı nedir, buradaki yapaylık nereden gelir?

Cevap: Öğrenciler bilgisayarın saatine bağlı olduğunu ve belli bir örüntü olduğunu söyledi. Hoca, gerçek anlamda rastgeleliğin hiçbir bağlayıcı etken olmadan değer üretmek olduğunu anlattı. Yazı tura için bile paranın dengeli olması, rüzgarın etki etmemesi gerekir. Rastgele sayı üreteçleri ise belli bir kök (seed) değerinden başlar ve belli bir uzunlukta sayı üretir.

Hocanın örneği şöyleydi. Kök değeriniz 5 olsun. Sonraki sayı bu 5'e bağlı olarak üretilir, örneğin 8. Ondan sonra örneğin 1000 üretilir. Devam edip bir yerde yeniden 5'e gelirseniz başa

dönmüş olursunuz, sonraki sayı yine 8 olacaktır. Bu yüzden bu sayılar tam rastgele değildir. Üreteçler ne kadar uzun bir tekrarla üretirse o kadar başarılıdır ve bunun farklı algoritmaları vardır.

> **Ders sonrası not:** Hoca "ne kadar uzun frekansta üretirlerse" dedi; doğrusu burada anlatılan kavram frekans değil, dizinin tekrara girene kadarki uzunluğu, yani periyottur.

`Random` kullanmak için sınıf import edilmelidir: `import java.util.Random;`. Hoca bir eksiği de vurguladı: Sınıfı import etmek onu kullanıma hazır hale getirmez. Metotlar `static` olmadığı için önce nesne üretmek gerekir. Öğrencilerden de bu cevap geldi. `Math` doğrudan kullanılabilir, `Random` ise import ile dolaylı olarak gelir.

```java
// Dosya: RastgeleDeneme.java
import java.util.Random;

public class RastgeleDeneme {
    public static void main(String[] args) {
        double m = Math.random();          // nesne yok, static metot
        Random r = new Random();           // nextInt static değil, nesne gerekir
        int a = r.nextInt();               // 2^32 olası int değerinden biri
        int b = r.nextInt(5);              // 0, 1, 2, 3 ya da 4
        double d = r.nextDouble();         // 0.0 ile 1.0 arası
        boolean e = r.nextBoolean();       // true ya da false

        System.out.println(m + " " + a + " " + b + " " + d + " " + e);
        System.out.println(Math.abs(a));
    }
}
```

Hocanın saydığı metotlar `nextBoolean`, `nextDouble`, `nextFloat`, `nextInt` ve `nextLong`'dur. `nextInt`'in iki aşırı yüklü sürümü üzerinde duruldu. Birincisi 2^{32} olası `int` değeri arasından düzgün dağılımlı bir sayı üretir. Python ile hesaplandı: $2^{32} = 4294967296$. İkincisi bir üst sınır (bound) alır. Hoca örnek olarak 5 verdi. Javac ile denendi: `r.nextInt(5)` bir milyon çağrıda en az 0, en çok 4 üretiyor, üst sınır dahil değildir. Daha büyük değerler için sonuç bir sayıyla çarpılabilir, toplanabilir ya da çıkarılabilir. Slayttaki son örnek de `Math.abs` ile rastgele bir sayının mutlak değerini almayı ve sayıyı parametreyle sınırlamayı gösteriyordu.

> **Ders sonrası not:** Hoca bu metotları sayarken "bunlar 0.0 ile 1.0 arasında kullanılır" dedi; doğrusu bu aralık `nextDouble` ve `nextFloat` için geçerlidir. Javac ile denendi: parametresiz `nextInt` negatif değerler de üretiyor. Ayrıca JDK 25'te yansıtma ile `nextInt`'in üçüncü sürümü, `nextInt(int, int)`, de görünüyor.

14. Çıktı: System.out

Komut satırına çıktı için `System.out` kullanılır. Hoca kaynak koda giderek gösterdi: `System` sınıfının içinde `out` adlı bir üye değişken vardır ve bu bir `PrintStream` nesnesidir. Bu sınıf, program ile işletim sistemi arasındaki iletişimi sağlar. Giriş için giriş akışı (`in`), çıkış için çıkış akışı (`out`), hata için hata akışı (`err`) vardır. Hata mesajları Java sanal makinesi tarafından hata akışına gönderilebilir. Hata akışı istisna konusunda görülecek.

`out` nesnesinin içinde `println` metodu vardır ve birçok aşırı yüklü sürümü bulunur. `print` ve `println`, C'deki `printf`'e benzer. Aralarındaki tek fark satır sonudur. Hoca ayrıca, uzun bir `System.out.println` içinde `+` koyup alt satırdan devam etmenin hiçbir engeli olmadığını söyledi.

> **Ders sonrası not:** Hoca burada "bu static bir tanımdır, bu metoda erişmek için nesne üretmek gerekmez" dedi; doğrusu `static` olan `println` değil, `System` sınıfının `out` alanıdır. `println`, `out` nesnesinin normal bir metodudur. Javac ile denendi: yansıtma ile `System.out` alanı `static`, `PrintStream.println` metodu `static` değil görünüyor. Nesneyi biz üretmeyiz, `out` nesnesi hazır gelir.

15. Girdi: Scanner ve nextLine Hatası

Java'da klavyeden girdi almak için `Scanner` sınıfı kullanılır. `new Scanner(...)` ile nesne üretilir ve parametre olarak `System.in` verilir, çünkü klavyeyi kullanmak için giriş akışı sağlanmalıdır. Sınıf tanımını görebilmek için `import java.util.Scanner;` eklenir. Aksi halde derleyici sınıfı bulamaz. Sonra `nextInt`, `nextFloat`, `nextLine` gibi birçok metotla istenen türde değer okunur.

Slayttaki örnek şöyleydi. Ad okunuyor, yaş okunuyor, kullanıcıya gelecek yılki yaşı söyleniyor, sonunda `close` çağrılıyor.

```
// Dosya: ScannerOrnek.java
import java.util.Scanner;

public class ScannerOrnek {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("What is your name? ");
        String name = in.nextLine();
        System.out.print("How old are you? ");
        int age = in.nextInt();
        System.out.println("Hello " + name + ". "
                + "Next year you will be " + (age + 1));
        in.close();
    }
}
```

Javac ile denendi: kod derleniyor. Ad ve 32 yaşı girildiğinde `Next year you will be 33` yazıyor. Hoca, `Scanner` işletim sistemiyle iletişim kurduğu için iş bitince kapatıldığını söyledi.

15.1 Hata: Girilemeyen Ad

Slayt `Scanner` sınıfında bir hatadan (bug) söz ediyordu: `nextInt`, `nextFloat` gibi bir metotla ilkel değer okuduktan sonra metin okunursa, metin boşa gider. Hoca sebebini öğrencilere sordu.

Soru: C'de bir sayı girip Enter'a bastıktan sonra karakter okunamıyordu. Hatırlıyor musunuz?

Cevap: Öğrencinin cevabı: `%c`'den önce bir boşluk bırakıyorduk, `scanf(" %c", ...)` gibi. Hoca bunun sebebini açıkladı: Bir önceki satırda girilen değerden sonra basılan Enter, tamponda (buffer) kalır. Sonraki okuma bunu alır ve klavyeden girdiğimiz şeyi göremeyiz.

Hoca aynı durumu önce yaşı, sonra adı okuyarak gösterdi.

```
// Dosya: ScannerHata.java
import java.util.Scanner;

public class ScannerHata {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("How old are you? ");
        int age = in.nextInt();
        System.out.print("What is your name? ");
        String name = in.nextLine();     // tampondaki satır sonunu okur
        System.out.println("[" + name + "] " + age);
        in.close();
    }
}
```

Hoca yaş olarak 32 girdi. Ad soruldu ama girilemeden program ilerledi. Javac ile denendi: Girdi olarak `32` ve sonra bir ad verildiğinde ad değişkeni boş kalıyor, çıktı `[] 32` oluyor.

Sebep adım adım şöyledir:

1. `nextInt` "bir tam sayı okuyana kadar" çalışır. 32 tam sayıya dahildir, Enter dahil değildir.
2. 32 alınır, Enter (satır sonu) tamponda kalır.

3. `What is your name?` ekrana yazılır.
4. `nextLine` tampona bakar ve orada bekleyen satır sonunu okur. Kullanıcı klavyeye basmadan bile okuma biter.

   Ders sonrası not: Hoca tamponda kalan şeye zaman zaman "boşluk" dedi; doğrusu orada kalan karakter satır sonudur. Slayt buna "bug" diyor, ama bu büyük olasılıkla bir hata değil `nextInt` ile `nextLine`'ın tanımlı davranışıdır. Javac ile denendi: aynı davranış JDK 25'te de birebir görülüyor.

15.2 Çözüm ve İkinci Deneme

Hoca, slaydın önerdiği çözümü "bug'ın etrafından dolaşmak" diye adlandırdı: tam sayıdan sonra boş bir `nextLine` çağrısı yapmak. Bir de arada ikinci bir tam sayı okumayı denedi. 12 ve 23 girdi.

```
// Dosya: ScannerCozum.java
import java.util.Scanner;

public class ScannerCozum {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("How old are you? ");
        int age = in.nextInt();
        System.out.print("How old are you (2)? ");
        int age2 = in.nextInt();         // önceki Enter'ı atlar
        in.nextLine();                   // kalan satır sonunu tüketir
        System.out.print("What is your name? ");
        String name = in.nextLine();     // artık gerçekten kullanıcıyı bekler
        System.out.println("[" + name + "] " + age + " " + age2);
        in.close();
    }
}
```

Javac ile denendi: Girdi `12`, `23` ve bir ad olduğunda ad artık okunuyor. Çıktıda ad ile birlikte `12 23` görünüyor.

Burada iki gözlem var. İkinci `nextInt`, birinciden kalan Enter'ı beyaz boşluk (whitespace) sayıp siler ve 23'ü alır. Ama 23'ten sonraki Enter yine tamponda kalır. Bu yüzden arada metin okurken dikkat etmek gerekir. Hoca bunun karmaşık bir şey olmadığını, ufak ve gözden kaçan bir durum olduğunu, çok önemli görmediğini ama anlatılması gerektiğini de ekledi.

Hoca C'deki `flush` fonksiyonunun Java'da olup olmadığına da baktı. `System.out` tarafında `flush` var, ama çıkışla ilgilidir. Bekleyen şey giriş akışındadır, çıkış bunu ilgilendirmez. Giriş akışında `skip` gibi metotlar gördü ve işe yarar bir yöntem bulamadı. Javac ile denendi: `InputStream` sınıfında `flush` metodu yok.

> **Ders sonrası not:** Hoca "C'de doğrudan flush işe yarıyor" dedi; bu, giriş akışı için büyük olasılıkla bazı derleyicilere özgü bir davranıştır. C standardına göre `fflush` büyük olasılıkla yalnız çıkış akışları için tanımlıdır. Bu bilgi altyazıdan ve denemeden doğrulanmadı.

16. Kendini Test Eden Sınıf: Car

Slayttaki `Car` örneğinde bir alan (`plate`), `getPlate` ve `setPlate` metotları, yapıcı ve aracı tanıtan bir metot vardı. Sınıfın içinde bir de `main` bulunuyordu. Kullanıcıdan plaka alınıyor, nesneye veriliyor, araç kendini tanıtıyor ve `Scanner` kapatılıyordu. Aşağıdaki kod anlatımdan yeniden kuruldu.

```
// Dosya: Car.java
import java.util.Scanner;

public class Car {
    private String plate;
```

```
    public Car(String plate) {
        this.plate = plate;
    }

    public String getPlate() {
        return plate;
    }

    public void setPlate(String plate) {
        this.plate = plate;
    }

    public void introduceSelf() {
        System.out.println("Plate: " + plate);
    }

    // Sınıf kendini test ediyor
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a license plate: ");
        Car car = new Car(input.nextLine());
        car.introduceSelf();
        input.close();
    }
}
```

Javac ile denendi: kod derleniyor ve girilen plaka ekrana yazılıyor.

Hoca, öğrencilerin burada şaşırdığını söyledi: `Car` sınıfını tasarladık, bu sınıf kendi içinden nasıl nesne üretir, nasıl referans tutar? Cevabı şudur: Bu kendini test eden bir sınıftır. Ayrıca bir test sınıfı yazmak yerine sınıfın içine `main` eklersiniz ve sınıf kendi kendini test eder. Bunda bir sıkıntı yoktur.

17. Metot Parametreleri: Değerle Aktarım

C'de program, büyük problemi küçük parçalara bölmek için birçok fonksiyona ayrılır. Bu modüler bir yaklaşımdır. Java'da da hem test sınıfında hem de kendi sınıflarımızda metotlar kullanılır.

Parametreler ilkel tipteyse aktarım değerle olur (call by value). Bu, C ya da C++'ta işaretçisiz parametre geçirmekle aynı şekilde çalışır. Hoca burada kendi düzeltmesini de yaptı: C'de parametre aktarımı varsayılan olarak değerledir. Java da değerle aktarımı kullanır. İlkel olmayan veri geçirilirken ise aktarılan şey referansın değeridir (call by value of references). Hoca bunu referansla aktarımın sahte bir biçimi, "yalancı bir call by reference" olarak adlandırdı.

> **Ders sonrası not:** Slayt anlatımında "Java mostly uses call by value" geçti; doğrusu Java'da aktarım her zaman değerledir. İlkellerde değer, nesnelerde referansın değeri kopyalanır. Hocanın ardından yaptığı açıklama da budur.

> **Hocanın vurgusu:** Hoca konunun önemini vurgulamak için şemayı yeniden çizdi. `main`'deki `Integer x`, `Integer y` ile `swap`'teki `Integer x`, `Integer y` ayrı değişkenlerdir. Fonksiyon içinde yeni referanslar oluşturulur ve eski referansların değeri bunlara kopyalanır. Bu yüzden değişim `main`'i etkilemez.

İşaretçilerde olduğu gibi bir değişiklik Java'da olmaz. Hoca, gerçek bir yer değiştirme için tek yolu şöyle özetledi: Bir sınıfla sarmalamak. Örneğin `Book` diye bir sınıfımız olsa onu da sarmalayan bir sınıf yazarız. İçindeki değer alınıp yedeklenir, sonra ilkel değerler üzerinden swap yapılır ve bu yukarıya yansır.

> **Ders sonrası not:** Hoca `MyClass` swap'ı için "deep copy" ya da "deep cloning" dedi; doğrusu burada nesne kopyalanmıyor. Aynı nesnelerin içindeki alan değerleri set

metoduyla değiştiriliyor. Javac ile denenen `SwapTest` örneğinde başka nesne üretilmiyor.

Slayttaki karşılaştırma şöyledir: C ya da C++'ta işaretçi olarak geçirilen parametreyle gerçek bellek konumuna erişilir ve swap kolayca yapılır. Java'da bu tür bir mekanizma yoktur. Referans aktarımı işaretçiye çok benzer, ama nesne parametrelerinin adreslerini değiştiremezsiniz. Bu yüzden nesne parametrelerindeki değişiklikler kalıcıdır, yalnız yeniden ilk değer verme ve nesneleri yer değiştirme bunun dışındadır. Hoca bunu şöyle açıkladı: Referanslar kopya olabilir, ama gösterdikleri yerler aynı kalır.

17.1 MethodParameterTest1 Örneği

Slayttaki sınıfta ilkel ve sarmalayıcı parametreli metotlar ile sınıf düzeyinde iki sarmalayıcı değişken vardı. Aşağıdaki kod anlatımdan yeniden kuruldu.

```
// Dosya: MethodParameterTest1.java
public class MethodParameterTest1 {
    static Integer wrapI = 1, wrapJ = 2;   // sınıf düzeyinde ("global") alanlar

    static void ilkelDuzenle(int x) {
        x++;
    }

    static void sarmalayiciDuzenle(Integer x) {
        x++;                               // x yeni bir Integer'ı gösterir
    }

    static void ilkelDegistir(int x, int y) {
        int temp = x;
        x = y;
        y = temp;
    }

    static void sarmalayiciDegistir(Integer x, Integer y) {
        Integer temp = x;
        x = y;
        y = temp;
    }

    static void globalDegistir() {
        Integer temp = wrapI;
        wrapI = wrapJ;
        wrapJ = temp;
    }

    public static void main(String[] args) {
        int count = 3;
        System.out.print(count + " -> ");
        ilkelDuzenle(count);
        System.out.println(count);         // 3 -> 3

        Integer wrapper1 = 5;
        System.out.print(wrapper1 + " -> ");
        sarmalayiciDuzenle(wrapper1);
        System.out.println(wrapper1);      // 5 -> 5

        int i = 1, j = 2;
        ilkelDegistir(i, j);
        System.out.println(i + " " + j);   // 1 2

        Integer a = 1, b = 2;
```

```
        sarmalayiciDegistir(a, b);
        System.out.println(a + " " + b);   // 1 2

        globalDegistir();
        System.out.println(wrapI + " " + wrapJ);   // 2 1
    }
}
```

Javac ile denendi: yorumlardaki sonuçlar çıkıyor. Derste hoca sonuçları şöyle okudu:

- `ilkelDuzenle(count)`: 3 kopyalanır, `x++` ile içeride 4 olur, ama `count` değişmez. Önce 3, sonra 3.
- `sarmalayiciDuzenle(wrapper1)`: 5 atanır, `x` artırılır, değişim yansımaz. 5 ve 5.
- `ilkelDegistir`: 1 ve 2, önce ve sonra aynı.
- `sarmalayiciDegistir`: yine 1 ve 2, önce ve sonra aynı. Hoca bunların metodun içinde swap gibi göründüğünü, ama çağıran metot için swap olmadığını söyledi.
- Sınıf düzeyindeki değişkenler: `wrapI` ve `wrapJ` her yerden yer değiştirilebilir. Önce ve sonra farklıdır.

  Ders sonrası not: Hoca, `sarmalayiciDuzenle`'nin yansımamasının sebebini ayrıca açıklamadı. Javac ile denendi: `Integer` sınıfında hiçbir `set` metodu yok, `x++` nesnenin içini değiştirmiyor, `x`'e yeni bir değer atıyor. Bu yüzden `wrapper1` 5 kalıyor.

Son satır gerçek bir swap sayılmaz. Hoca, bu değişkenlerin zaten global olduğunu, sınıf üyelerine erişmekle aynı şey olduğunu hatırlattı. Kitap örneğindeki `id`, `ad`, `yazar` alanlarına erişmek de böyledir. Yapılabilir, ama gereği yoktur. Global değişken kullanılacaksa global tam sayı da kullanılabilirdi. Yalnız bizim yaptığımız `MyClass` yöntemi gerçek anlamda işe yarar. Normal bir tam sayı için yapılmaz ama kitap sınıfı gibi bir şey için gerekirse yapılabilir.

17.2 Metot İçinde Önce ve Sonra

Aradan sonra hoca, kendi deyişiyle faydasız bir slayt örneğine baktı. Metot içinde değişiklikten önce ve sonra değer yazdırılıyordu.

```
// Dosya: IcerdeDegisim.java
public class IcerdeDegisim {
    static void degistir(int x, int y) {
        System.out.println("önce:  " + x + " " + y);
        int t = x;
        x = y;
        y = t;
        System.out.println("sonra: " + x + " " + y);
    }

    static void degistir(Integer x, Integer y) {
        System.out.println("önce:  " + x + " " + y);
        Integer t = x;
        x = y;
        y = t;
        System.out.println("sonra: " + x + " " + y);
    }

    public static void main(String[] args) {
        degistir(1, 2);
        degistir(Integer.valueOf(3), Integer.valueOf(5));
    }
}
```

Javac ile denendi: Çıktı önce `1 2`, sonra `2 1`, ardından `3 5`, sonra `5 3`. Sonuç hem ilkel hem sarmalayıcı için aynıdır. Hocanın yorumu şuydu: Burada metottan önce ve sonra diye bir şey yok, her şey metodun içinde oluyor. İçeride önceki ve sonraki değerin farklı olması algoritmanın doğal akışıdır. Aynı olmasını beklemek zaten garip olurdu.

Slaytlardaki görsel programlama kısmı için geçen soruda hoca, GUI'nin Graphical User Interface demek olduğunu ve Swing, JavaFX gibi kitaplıklarla yapılan görsel programlamayı ifade ettiğini söyledi.

18. Dizileri Metoda Göndermek

Dizi bir metoda ya da sınıfa gönderildiğinde gerçek değerler değişir. Bu, C'dekine benzer. C'de diziyi fonksiyona geçirirken aslında dizinin adresini, yani ilk elemanının adresini geçirirsiniz. Bu yüzden dizideki değişiklik çağıran fonksiyona yansır.

Slayttaki örnekte beş elemanlı bir tam sayı dizisi vardı. Önce dizi yazdırılıyor, sonra `static` olan `lowHighSwap` metodu sınıf adı üzerinden çağrılıyordu. Metot, ilk eleman ile son elemanı yer değiştiriyordu.

```
// Dosya: DiziAktarma.java
public class DiziAktarma {
    static void lowHighSwap(int[] dizi) {
        int temp = dizi[0];
        dizi[0] = dizi[dizi.length - 1];
        dizi[dizi.length - 1] = temp;
    }

    public static void main(String[] args) {
        int[] d = {1, 2, 3, 4, 5};
        for (int i = 0; i < d.length; i++)
            System.out.print(d[i] + " ");
        System.out.println();

        DiziAktarma.lowHighSwap(d);       // static: sınıf adıyla çağrılır

        for (int i = 0; i < d.length; i++)
            System.out.print(d[i] + " ");
        System.out.println();
    }
}
```

Javac ile denendi: Çıktı `1 2 3 4 5` ve `5 2 3 4 1`. Yani orijinal veri değişti.

> **Hocanın vurgusu:** Hoca, kitaplardaki "metottan önce/sonra yazdır" örneklerini laf kalabalığı olarak eleştirdi. Önemli olan, bir verinin fonksiyona girip o fonksiyondan değişmiş olarak çıkmasıdır.

19. Karar ve Döngü Yapıları

Akış kontrolünün C'den farkı yoktur. Hoca bunları yapısal programlama dersinden bilindiği için hızlı geçti.

- `if`, `else`: küme parantezleriyle C'deki gibidir.
- Karşılaştırma işleçleri: küçük, büyük, küçük eşit, büyük eşit, eşit, eşit değil.
- Mantıksal işleçler: Birden fazla koşulu tek ifadede birleştirmek için mantıksal ve (&&) ile mantıksal veya (||) kullanılır. Slayttaki örnekte iki koşul birlikte denetlenip buna göre harf notu değerlendirmesine geçiliyordu.
- `for`, `while`, `do-while`: C ile aynıdır. `do-while` koşulundan sonra noktalı virgül gerekir.
- `switch`: C ile aynıdır.

20. for-each Döngüsü

Bu döngülere ek olarak bir de `for-each` vardır. Hoca bunu, bir öğrencinin önceki sorusuna cevap olarak anlattı. Dizi ya da liste biçiminde bir yapınız varsa normal `for` ile de gezebilirsiniz, ama bu daha özel bir yazımdır. Hoca C#'ta da aynısının olduğunu söyledi. C#'ta `in` yazılır, Java'da iki nokta konur.

```java
// Dosya: ForEachDeneme.java
public class ForEachDeneme {
    public static void main(String[] args) {
        int[] dizi = {1, 2, 3, 4, 5};
        int toplam = 0;
        for (int a : dizi) {
            System.out.print(a + " ");
            toplam += a;
            a = a + 1;                // yalnız kopya değişir
        }
        System.out.println();
        System.out.println("Toplam: " + toplam);
        System.out.println("İlk eleman: " + dizi[0]);
    }
}
```

Derste sonuç “1, 2, 3, 4, 5, toplamda 15” oldu. Javac ile denendi, aynı sonuç çıkıyor. İlk eleman da 1 kalıyor.

Bu döngü ilk elemandan başlar. Hiç atlamadan son elemana kadar bütün elemanları sırayla bir defa değişkene kopyalar. İçeride değerleri toplayabilir ya da ekrana yazdırabilirsiniz. Hoca bunun aslında bir yineleyici (iterator) gibi olduğunu söyledi.

Döngü içinde `a = a + 1` yapılırsa dizideki değer değişmez. Çünkü eleman `a`’nın içine kopyalanmıştır. Hoca bunu bir okuma tarzı, “salt okunur gibi” diye tarif etti.

Soru: `for-each` nesnelerle çalışır mı? Örneğin kitap dizisiyle?

Cevap: Evet, olur. Sonraki bölümde bunun kodu yazıldı.

21. String ile Swap ve StrClass

Slayttaki bir sonraki örnekte `f` fonksiyonu bir metni alıyor, büyük ya da küçük harfe çeviriyor, önce ve sonra halini yazdırıyor, sonra sonucu döndürüyordu. Hoca bundan sonra asıl konuya geldi. `g` fonksiyonu iki `String` alıp yer değiştirmeye çalışıyordu. `String` biraz önceki `Integer`’a benzer. `g`’den önceki ve sonraki sonuçlar aynı kalır.

Gerçekten swap isteniyorsa hoca `StrClass` adında bir sınıf yazdı. Sınıf içinde tek bir `String` tutuyor ve `public` bırakılıyor. Hoca bunu sarmalayıcı sınıfların faydası olarak gösterdi. Aşağıdaki kod anlatımdan yeniden kuruldu, `f` yalnız küçük harfe çevirecek biçimde sadeleştirildi.

```java
// Dosya: StringSwap.java
class StrClass {
    public String s;

    public StrClass(String s) {
        this.s = s;
    }
}

public class StringSwap {
    static String f(String a) {
        System.out.println("f içinde önce:  " + a);
        a = a.toLowerCase();
        System.out.println("f içinde sonra: " + a);
        return a;
    }

    static void g(String x, String y) {
        String t = x;
        x = y;
        y = t;
    }
```

```
    static void h(StrClass x, StrClass y) {
        String t = x.s;
        x.s = y.s;
        y.s = t;
    }

    public static void main(String[] args) {
        System.out.println(f("MERHABA"));

        String a = "merhaba", b = "dünya";
        System.out.println("before g: " + a + " " + b);
        g(a, b);
        System.out.println("after g:  " + a + " " + b);

        StrClass s1 = new StrClass("merhaba");
        StrClass s2 = new StrClass("dünya");
        System.out.println("before h: " + s1.s + " " + s2.s);
        h(s1, s2);
        System.out.println("after h:  " + s1.s + " " + s2.s);
    }
}
```

Derste `g` için `merhaba dünya`, `merhaba dünya` çıktı, hiçbir değişim olmadı. `h` için önce `merhaba dünya`, sonra `dünya merhaba` çıktı. Javac ile denendi, aynı sonuçlar çıkıyor.

22. Nesne Dizisinde for-each: Kütüphane Örneği

Hoca, ikinci haftanın uygulamasında yapılan kütüphane örneğine döndü ve öğrencinin sorusunu burada cevapladı. Elimizde `Kutuphane`, `Kitap` ve `Yazar` sınıfları vardı. `Kutuphane` nesnesinin içinde kitaplardan oluşan bir dizi vardı.

`for-each` için kütüphane nesnesinin kendisi kullanılamaz. Ama içindeki kitaplar dizisi kullanılabilir. Bu bir dizidir ve her elemanı bir kitaptır. Döngü değişkeni `Kitap` türünden `k`'dir. Her turda `k` bir nesne olur, ilk turda sıfırıncı kitap alınır. `Kitap` içindeki `kitapTanit` metodu da değer döndürmeyip ekrana yazdığı için doğrudan çağrılır.

```
for (Kitap k : kutuphane.kitaplar)
    k.kitapTanit();
```

Hoca burada bir hatırlatma yaptı. Kitaplar dizisine doğrudan erişebiliyoruz, çünkü bu örnekte daha genel yazmıştık. Dizi `private` olsaydı get ve set gibi bir yöntem kullanacaktık. Çalıştırılınca kütüphane listeleme metodunun çıktısı ile `for-each` çıktısının birebir aynı olduğu görüldü. Araya bir ayraç yazısı konarak bu karşılaştırıldı. Böylece soru cevaplanmış oldu.

23. toString Metodunu Ezmek

Hoca bir soru daha sordu: `k`'yı doğrudan yazdırmak istersem ne olur? Ekranda kitap bilgileri değil, adres gibi bir şey göründü. Peki kitap yazdırınca az önce gördüğüm mesajları göremez miyim?

Bir nesne yazdırılırken aslında bir metot çalışır. Bu metodun adı `toString`'dir. Nesneden sonra nokta koyunca çıkan listede `toString`, `equals`, `getClass`, `hashCode` görünür. `toString`'i kendi amacımıza göre değiştirirsek gördüğümüz çıktı da değişir.

Soru: Bu `toString` nereden çıktı? Hiçbir yerde tanımlamadık, ama her nesnede var.

Cevap: `Object` sınıfından geliyor. `Object` bütün sınıfları kapsayan bir sınıftır ve `toString` orada tanımlanmıştır. Tanımı şunu der: paketin adını yaz, altındaki sınıfın adını yaz, bir de bellekteki referans adresini yaz.

> **Ders sonrası not:** Hoca varsayılan çıktının son kısmına "bellekteki referans adresi" dedi; doğrusu `@` işaretinden sonraki kısım nesnenin `hashCode` değerinin onaltılık ya-

zılışıdır. Javac ile denendi: `Object.toString()` sonucu `sınıf adı + "@" + Integer.toHexString(hashCode())` ile birebir aynı çıkıyor.

Hoca önce küçük bir deneme yaptı. `Kitap` içinde `toString`'i `this.getClass()` çıktısının yanına bir metin ve `null` ekleyerek yazdı. Ekranda paket ve sınıf adı, yanında eklediği metin ve `null` göründü. Hepsi tek bir metin olarak gelmişti. `null` bir metnin yanına gelince metne dönüşür. Javac ile denendi: `getClass().getName() + " kitap" + null` sonucu `Kitap kitapnull` oluyor (paketsiz bir sınıfta).

23.1 Kitap ve Yazar İçin toString

Sonra hoca `kitapTanit` içindeki ifadeyi `toString`'e taşıdı. Dönüş tipi `String` olduğu için ekrana yazmak yerine bir `s` değişkeninde metin biriktirdi. Yazar `null` ise `s`'ye "yazar bulunamadı" ekledi. Değilse yazarın bilgisini ekledi. En sonda `s`'yi döndürdü.

Hoca ardından sordu: Kitapta yapabildiysem yazarda da yapabilir miyim?

Soru: `toString`'i `Yazar` sınıfında da ezebilir miyiz?

Cevap: Evet. `toString` `Object` sınıfından geldiğine göre `Yazar`'a da gelir. Hiyerarşi düşünülürse hem `Kitap`'ın hem `Yazar`'ın üstünde `Object` vardır. Nitekim `y1.` yazınca listede `toString` görünür. Hangi nesneyi üretirseniz üretin `toString` vardır.

Kod anlatımdan yeniden kuruldu. Derste daha fazla kitap ve yazar vardı, burada iki kitapla kısaltıldı. Metin biçimleri örnek amaçlıdır.

```
// Dosya: KutuphaneTest.java
class Yazar {
    private int id;
    private String ad;

    public Yazar(int id, String ad) {
        this.id = id;
        this.ad = ad;
    }

    @Override
    public String toString() {
        return "Yazar ID:" + this.id + " Yazar Adı:" + this.ad;
    }
}

class Kitap {
    private int id;
    private String ad;
    private Yazar yazar;

    public Kitap(int id, String ad, Yazar yazar) {
        this.id = id;
        this.ad = ad;
        this.yazar = yazar;
    }

    public void kitapTanit() {
        System.out.println(this);    // this.toString() çalışır
    }

    @Override
    public String toString() {
        String s = "ID:" + this.id + " Ad:" + this.ad;
        if (this.yazar == null)
            s += " Yazar bulunamadı";
        else
```

```
            s += " " + this.yazar;   // Yazar'ın toString'i çalışır
        return s;
    }
}

class Kutuphane {
    public Kitap[] kitaplar;         // örnekte erişilebilir bırakıldı
    private int adet = 0;

    public Kutuphane(int kapasite) {
        kitaplar = new Kitap[kapasite];
    }

    public void kitapEkle(Kitap k) {
        kitaplar[adet++] = k;
    }

    public void listele() {
        for (int i = 0; i < adet; i++)
            kitaplar[i].kitapTanit();
    }
}

public class KutuphaneTest {
    public static void main(String[] args) {
        Kutuphane kutuphane = new Kutuphane(2);
        kutuphane.kitapEkle(new Kitap(1, "Kitap1", null));
        kutuphane.kitapEkle(new Kitap(2, "Java1", new Yazar(1, "Yazar1")));

        kutuphane.listele();
        System.out.println("*****");
        for (Kitap k : kutuphane.kitaplar)
            k.kitapTanit();
        System.out.println("*****");
        for (Kitap k : kutuphane.kitaplar)
            System.out.println(k);
    }
}
```

Javac ile denendi: Her üç blokta da şu iki satır çıkıyor:

```
ID:1 Ad:Kitap1 Yazar bulunamadı
ID:2 Ad:Java1 Yazar ID:1 Yazar Adı:Yazar1
```

Hoca kodda `this.yazar.toString()` ve `this.yazar` yazımlarının aynı işi gördüğünü söyledi. `this.yazar` yazılsa da gidip `toString` çağrılır. Yani iç içe metotlar çağrılır: `Kitap`'ın `toString`'i `Yazar`'ın `toString`'ini çağırır. Metodu çağırırken adını yazmaya da gerek yoktur. Nesne referansıyla yazdırınca `toString` otomatik çağrılır. Hoca, derleme yapılmadığı için bir an `toString`'i listede göremedi. Sonra kodu çalıştırdı. "Yazar bulunamadı" yerine artık yazar bilgisi göründü. `toString`, önceki alt alta yazma yerine bilgileri tek metinde birleştirdi.

23.2 Override Ne Demek?

`Yazar` için `toString` yazınca NetBeans "Add @Override annotation" önerisini verdi. Hoca bunu fırsat bilerek açıkladı. Override, bir şeyin üstüne yazmaktır. `Object` sınıfında tanımlanmış `toString`'i ezmiş olduk, yani onun tanımlama yetkisini kendi elimize aldık. Hoca, metoda üst sınıftaki uygulamaya gitme komutuyla bakınca `Object` sınıfına gelindiğini de gösterdi. Derlenmiş kaynak gizlendiği için içi görünmüyordu. Ama metot `public` tanımlanmış olduğu için `Yazar`, `Kitap` ya da herhangi bir sınıfta kullanabiliyoruz.

> **Ek bilgi (derste anlatılmadı):** `@Override` zorunlu değildir, ama derleyiciye "bu metot üst sınıftaki bir metodu eziyor" der. Ad yanlış yazılırsa hata yakalanır. Javac ile

denendi: `@Override public String tostring()` yazınca `method does not override or implement a method from a supertype` hatası veriliyor. İşareti yazmayıp adı yanlış yazmak ise derleme hatası vermiyor, varsayılan `toString` çalışmaya devam ediyor.

Hoca bu konuya kalıtımda daha ayrıntılı dönüleceğini söyledi.

Dersin Toparlaması

Bu derste hoca birkaç ufak ama temel konuyu birleştirdi. Paketlerin hiyerarşik olduğunu ve yıldızın alt paketleri almadığını gördük. Bilgi gizlemenin ve UML yazım düzeninin nedenlerini tekrar ettik. İlkel tiplerin sarmalayıcı sınıflarını, `String` metotlarını, `Math` sınıfındaki `static` metotları ve nesne gerektiren `Random` sınıfını öğrendik. `Scanner` ile tamponda kalan satır sonunun yol açtığı `nextLine` sorununu ve çözümünü gördük.

Dersin omurgası parametre aktarımıydı. Java'da aktarım her zaman değerledir. Nesnelerde kopyalanan şey referansın değeridir. Bu yüzden `Integer` ve `String` ile yazılan swap çalışmaz. Ama kendi sınıfımızla, aynı nesnenin içeriği değiştirilirse değişiklik kalıcı olur. Diziler de bu yüzden metot içinden değişebilir. `for-each` ise dizideki değerlerin kopyasıyla çalışır. Son olarak bütün sınıfların `Object` sınıfından aldığı `toString` metodunu ezerek kalıtım konusuna bir giriş yapıldı. Ezme konusuna kalıtımda, aşırı yüklemedeki belirsizliğe de metot aşırı yükleme konusunda dönülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.